```
#!/usr/bin/env python
"""Upload approved clips from ready/<date>/ to the operator's own YouTube channe
l.

Complete, working upload flow (YouTube Data API v3, videos.insert, resumable upl
oad):

  1. First run opens a browser for OAuth consent on the operator's own Google ac
count
     (scope: youtube.upload only). The refresh token is stored locally in data/y
outube_token.json.
  2. Each approved clip in ready/<date>/ is a pair: NAME.mp4 + NAME.txt (title,
caption, tags,
     written by the review page when the operator approves the clip).
  3. videos.insert uploads the file with that metadata. Privacy defaults to "pri
vate"; pass
     --privacy public, or --publish-at to schedule. The video ID and URL are wri
tten back to
     NAME.txt and logged, so nothing is uploaded twice.

Usage:
  python scripts/youtube_upload.py                       # upload today's approve
d clips as private
  python scripts/youtube_upload.py --privacy public
  python scripts/youtube_upload.py --publish-at "2026-10-27T16:00:00Z" --date 20
26-10-27
  python scripts/youtube_upload.py --limit 1 --dry-run

Setup (once): in Google Cloud console enable "YouTube Data API v3", create an OA
uth client ID of
type "Desktop app", download it as data/youtube_client_secret.json.
"""
from __future__ import annotations

import argparse
import json
import re
import sys
import time
from datetime import date
from pathlib import Path

from google.auth.transport.requests import Request
from google.oauth2.credentials import Credentials
from google_auth_oauthlib.flow import InstalledAppFlow
from googleapiclient.discovery import build
from googleapiclient.errors import HttpError
from googleapiclient.http import MediaFileUpload

ROOT = Path(__file__).resolve().parent.parent
SCOPES = ["https://www.googleapis.com/auth/youtube.upload"]
CLIENT_SECRET = ROOT / "data" / "youtube_client_secret.json"
TOKEN = ROOT / "data" / "youtube_token.json"
RETRIABLE = (500, 502, 503, 504)


def youtube_client():
    creds = None
    if TOKEN.exists():
        creds = Credentials.from_authorized_user_file(str(TOKEN), SCOPES)
    if not creds or not creds.valid:
        if creds and creds.expired and creds.refresh_token:
```

```
            creds.refresh(Request())
        else:
            if not CLIENT_SECRET.exists():
                sys.exit(f"Missing {CLIENT_SECRET}. Create a Desktop OAuth clien
t in Google Cloud and save it there.")
            creds = InstalledAppFlow.from_client_secrets_file(str(CLIENT_SECRET)
, SCOPES).run_local_server(port=0)
        TOKEN.write_text(creds.to_json())
    return build("youtube", "v3", credentials=creds)


def read_meta(txt: Path) -> dict:
    """Parse the NAME.txt the review page writes on approval:

        TITLE:
        <title>

        DESCRIPTION / CAPTION:
        <caption, may span lines>
        <required tags>

        Credit: ...
        Posted: <url>      (added by this script after upload)
    """
    body = txt.read_text()
    meta = {"title": txt.stem, "description": "", "tags": [], "posted": ""}
    m = re.search(r"^TITLE:\s*\n(.+?)\n", body, re.M)
    if m:
        meta["title"] = m.group(1).strip()[:100]
    m = re.search(r"^DESCRIPTION / CAPTION:\s*\n(.*?)(?=\n\s*\n(?:Credit:|SOURCE
:|PERMISSION:|CHECKLIST:)|\Z)", body, re.M | re.S)
    if m:
        meta["description"] = m.group(1).strip()[:5000]
    meta["tags"] = sorted({t.lstrip("#") for t in re.findall(r"#\w+", meta["desc
ription"])})[:30]
    m = re.search(r"^Posted:\s*(\S+)", body, re.M)
    if m:
        meta["posted"] = m.group(1)
    return meta


def upload(yt, video: Path, meta: dict, privacy: str, publish_at: str | None) ->
 str:
    status = {"privacyStatus": privacy, "selfDeclaredMadeForKids": False}
    if publish_at:
        status["privacyStatus"] = "private"   # YouTube requires private + publi
shAt for scheduling
        status["publishAt"] = publish_at
    body = {
        "snippet": {"title": meta["title"], "description": meta["description"],
"tags": meta["tags"],
                    "categoryId": "24"},  # Entertainment
        "status": status,
    }
    media = MediaFileUpload(str(video), chunksize=8 * 1024 * 1024, resumable=Tru
e, mimetype="video/mp4")
    request = yt.videos().insert(part="snippet,status", body=body, media_body=me
dia)
    response, retry = None, 0
    while response is None:
        try:
            progress, response = request.next_chunk()
```

```
            if progress:
                print(f"    {int(progress.progress() * 100)}%", end="\r")
        except HttpError as e:
            if e.resp.status in RETRIABLE and retry < 5:
                retry += 1
                wait = 2 ** retry
                print(f"    retriable error {e.resp.status}; retrying in {wait}s
")
                time.sleep(wait)
            else:
                raise
    return response["id"]


def main() -> None:
    p = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.Ra
wDescriptionHelpFormatter)
    p.add_argument("--date", default=date.today().isoformat(), help="ready/<date
> folder to upload (default today)")
    p.add_argument("--privacy", choices=["private", "unlisted", "public"], defau
lt="private")
    p.add_argument("--publish-at", help="RFC 3339 UTC time, e.g. 2026-10-27T16:0
0:00Z (schedules a private video)")
    p.add_argument("--limit", type=int, default=10)
    p.add_argument("--dry-run", action="store_true")
    args = p.parse_args()

    folder = ROOT / "ready" / args.date
    pairs = [(v, v.with_suffix(".txt")) for v in sorted(folder.glob("*.mp4")) if
 v.with_suffix(".txt").exists()]
    if not pairs:
        print(f"Nothing to upload in {folder}")
        return
    yt = None if args.dry_run else youtube_client()
    done = 0
    for video, txt in pairs:
        meta = read_meta(txt)
        if meta["posted"]:
            print(f"  skip (already posted): {video.name} -> {meta['posted']}")
            continue
        print(f"  uploading {video.name}: {meta['title']!r} [{args.privacy}{' sc
heduled' if args.publish_at else ''}]")
        if args.dry_run:
            continue
        vid = upload(yt, video, meta, args.privacy, args.publish_at)
        url = f"https://youtube.com/shorts/{vid}"
        txt.write_text(txt.read_text().rstrip() + f"\nPosted: {url}\n")
        with (ROOT / "data" / "uploads.log").open("a") as log:
            log.write(json.dumps({"date": args.date, "file": video.name, "video_
id": vid, "url": url,
                                  "privacy": args.privacy, "publish_at": args.pu
blish_at}) + "\n")
        print(f"    done -> {url}")
        done += 1
        if done >= args.limit:
            break
    print(f"{done} uploaded.")


if __name__ == "__main__":
    main()
```